



// session 02 beginner Python

Input & conditionals

Give your crew a voice. Make the game respond.



Your ship changed when a **variable** changed.

Turn to a partner: name one value you personalised and the type of data it stored.

ship name → string

oxygen → integer

speed → float

// the problem watch the game



The ship still flies itself.

arrive



engine decides



fly on

Where should the player get a choice?

The answer lands in a variable.

Trace the lines from top to bottom.

```
name = input("What is your name? ")  
print("Hello, " + name)
```

OUTPUT What is your name? Aisha
Hello, Aisha



```
// predict before we run
```

If the player types **17**, what type is age?

integer

float

string

```
// reveal input returns text
```

`input()` gives us a **string**.

Even digits arrive as text. We will convert them later.

```
age = input("How old are you? ")  
print(type(age).__name__)
```

OUTPUT How old are you? 17
str

if chooses a path.

Python checks one question. A true answer runs the first block; otherwise it runs `else`.

condition is True

run the `if` block

condition is False

run the `else` block

Only one branch runs.

With the fixed answer "y", predict the printed sentence.

```
answer = input("Do you like pizza? (y/n) ")

if answer == "y":
    print("Great choice.")
else:
    print("More for me.")
```


// precision one sign or two?

= stores. == compares.

Read line two as: "Is x equal to 5?"

```
x = 5          # assign 5 to x
print(x == 5)  # compare x with 5
```

= put a value in

= ask a question

Spaces draw the boundary.

The two indented lines belong to the `if`. The final line does not.

```
answer = input("Launch? (y/n) ")

if answer == "y":
    print("Engines on")
    print("This is inside the if")
print("This always runs")
```

elif checks another choice.

Python stops at the first condition that is true.

```
mood = input("How are you feeling? ")

if mood == "great":
    print("Let's launch!")
elif mood == "tired":
    print("Rest first, captain.")
else:
    print("Let's talk about it.")
```



// predict trace the branches

**The player types "maybe".
Which branch runs?**

if

elif

else

```
// answer neither condition matches
```



else runs.

It catches every answer not already handled.

great? no tired? no **else** ✓

The engine asks one question.

We will fill in a small hook called `should_stop()`. Read def as “make this named block.”

ENGINE

Should we stop?

YOUR CODE

True or False

ENGINE

Acts on the answer

01 / ask

Capture the answer.

Add this above `main()` in `game.py`.

```
def should_stop():  
    answer = input(" Stop here? (y/n): ")
```


02 / answer

Hand a decision back.

return means “give this answer to the engine.”

```
def should_stop():  
    answer = input(" Stop here? (y/n): ")  
    if answer == "y":  
        return True  
    else:  
        return False
```


Pass the hook to `travel()`.

Add `should_stop` after the ship name. No brackets here—the engine will call it.

```
travel(  
    galaxy,  
    constants.STARTING_OXYGEN,  
    constants.STARTING_HULL,  
    constants.SHIP_NAME,  
    should_stop,  
)
```

Make every destination a decision.

1 Add `should_stop()` above `main()`.

2 Pass it into `travel()`.

3 Run `uv run game.py`.

4 Try both `y` and `n`.

Success looks like

The game pauses at every destination and obeys both choices.

20-25 min

Test like a developer.

One successful run is not enough. Prove each path works.

TEST 1

Type y

An encounter runs.

TEST 2

Type n

The ship flies past.

TEST 3

Mix choices

The game keeps asking.

Two likely mistakes. Two useful clues.

SyntaxError

Check `=` and colons.

A condition needs two equals signs and each branch header ends with `:`.

IndentationError

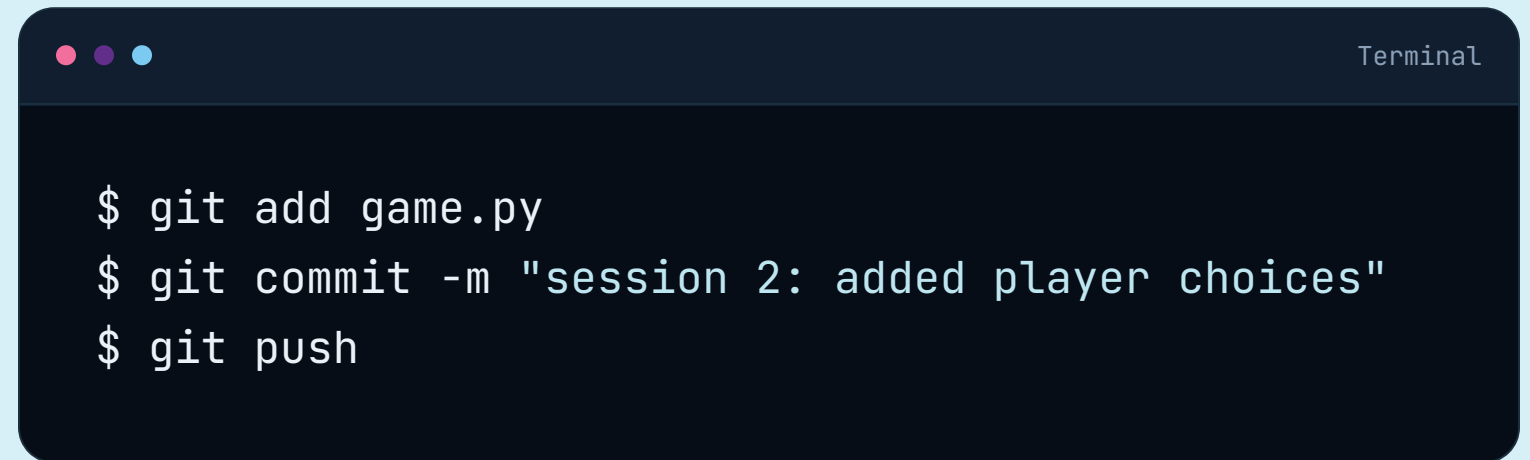
Align the blocks.

Lines inside each branch use four spaces. `e`lse lines up with `i`f.

```
// save git checkpoint
```

Save the moment the game learned to listen.

Run the game once more before committing.



```
$ git add game.py  
$ git commit -m "session 2: added player choices"  
$ git push
```



// reflect say it plainly

How does the game choose what happens next?

- 01 What does `input()` give back?
- 02 When does an `else` block run?
- 03 What is the difference between `=` and `==`?

Make the captain more forgiving.

Choose one only after your core build is committed.

EASY

Accept yes

Add another accepted answer.

MEDIUM

Handle surprises

Use `elif` for `n`, then explain the fallback.

STRETCH

Show resources

Print useful state before the prompt.



// mission complete session 02

Your ship now listens before it acts.

Next: replace the engine's galaxy with a list of your own worlds.

Session 03 • Lists